BML MUNJAL UNIVERSITY

School of Engineering and Technology

Course: MTH2802 – Linear Algebra & Differential Equations

B.Tech Computer Science & Engineering, Semester III

Project Report

Predicting Population Movement Using Transition Matrices

A Markov-chain model of movement between Delhi, Gurugram, Noida and Faridabad

Group 14

Name	Roll No.
Mansi Awasthi	250450
Prince Kumar Yadav	250500
Shobhit Rai	250557
Ishank Chauhan (Group Representative)	250406

Instructor: Dr. Luckshay Batra

November 2026

Abstract

In this project we model how people move between four regions of the Delhi–NCR area (Delhi, Gurugram, Noida and Faridabad) using a column-stochastic transition matrix P, with one time step equal to one year and the update rule $x_{k+1} = P\ x_k$. The starting populations are approximate Census 2011 figures, while the transition probabilities are **simulated** because public inter-city annual flow data are not available at this level. We verify that P is regular, compute future distributions through matrix powers, and find the long-term (steady-state) distribution in three independent ways: the eigenvector for $\lambda = 1$, Gaussian elimination on $(P - I)\pi = 0$, and a high matrix power. All three agree to within 10^{-8}. The model gives the steady state Delhi 55.5%, Gurugram 17.5%, Noida 16.7% and Faridabad 10.3%. All eigenvalues are real and distinct (1, 0.8862, 0.8638, 0.8600), so P can be diagonalised, and the second-largest eigenvalue 0.8862 explains the convergence speed (about 32 years to come within 1%). We also study two what-if scenarios and a 1000-run sensitivity test, and we discuss the limits of the model honestly.

1 Introduction and Problem Statement

People in a metropolitan region keep moving between cities for work, housing and family reasons. If, each year, a fixed fraction of the people living in one city move to another, then the future population of every city can be predicted with simple matrix multiplication. This is the idea of a **transition matrix**, and it is a good first application of the linear algebra we study in MTH2802.

The official brief for this project is: construct a transition matrix representing movement of people between selected regions using simulated or publicly available data; use matrix powers to estimate future population distributions and study the long-term behaviour of the model. We choose four regions of the National Capital Region (NCR): Delhi, Gurugram, Noida and Faridabad.

1.1 Objectives

1. Construct a 4×4 column-stochastic transition matrix P for the four regions and verify its properties.
2. Use repeated multiplication and matrix powers P^k to project the population for $k = 0$ to 30 years.
3. Find the long-term steady state in three independent ways and check that they agree.
4. Diagonalise P ($P = V D V^{-1}$) and use the eigenvalues, especially $|\lambda_2|$, to explain how fast the model converges.
5. Study scenarios (what-if changes) and the sensitivity of the answer to small errors in P.
6. State the limitations of the model, including what happens when P is not regular.

1.2 Convention used

We follow the textbook of Poole and use **column-stochastic** matrices. The entry P[i][j] is the probability that a person living in region j this year lives in region i next year, so every column sums to 1. If x_k is the population vector after k years, then $x_{k+1} = P\ x_k$. The order of regions is always Delhi, Gurugram, Noida, Faridabad.

2 Mathematical Background

2.1 Stochastic matrices and Markov chains

A square matrix P is **column-stochastic** if all its entries are non-negative and every column sums to 1. A **Markov chain** is a process that moves between a finite number of states where the probability of the next state depends only on the present state (the *memoryless* property). Here the states are the four regions and the transition probabilities are stored in P.

Why columns sum to 1. Column j lists what happens to a person living in region j: they must live somewhere next year, either in the same region or in one of the others. The probabilities of all these possibilities add up to 1. Consequently the total population is unchanged by $x_{k+1} = P\,x_k$ (our model is a closed system: nobody is born, dies, enters or leaves the four regions).

2.2 Why λ = 1 is always an eigenvalue

Let **1** be the vector of all ones. Since each column of P sums to 1, we have $\mathbf{1}^T P = \mathbf{1}^T$, which means $P^T\mathbf{1} = \mathbf{1}$. So 1 is an eigenvalue of P^T. A matrix and its transpose have the same characteristic polynomial because $\det(P - \lambda I) = \det(P^T - \lambda I)$, so 1 is also an eigenvalue of P. Moreover, every eigenvalue λ of a stochastic matrix satisfies $|\lambda| \le 1$.

A useful by-product: if v is an eigenvector of P for an eigenvalue $\lambda \neq 1$, then $\mathbf{1}^T P v = \mathbf{1}^T v$ gives $(1 - \lambda)(\mathbf{1}^T v) = 0$, so the entries of v add up to 0. Eigenvectors for $\lambda \neq 1$ therefore describe *redistributions* of population, not population itself.

2.3 Regular chains and the steady-state theorem

P is **regular** if some power P^k has all entries strictly positive (i.e. after k steps it is possible to get from any region to any other). **Steady-state theorem (stated, not proved):** if P is regular, then (i) λ = 1 is a simple eigenvalue and all other eigenvalues satisfy $|\lambda| < 1$; (ii) there is a unique probability vector π with $P\pi = \pi$; and (iii) for any starting distribution x_0 with total 1, $x_k = P^k x_0 \to \pi$ as $k \to \infty$. Moreover P^k approaches the matrix whose every column is π.

Intuition: if every region can reach every other region, the process keeps mixing the population. Memory of the starting point fades and only the balance between inflow and outflow remains. At the steady state, the number leaving each region equals the number arriving, so the shares stop changing.

2.4 Diagonalisation and P^{n} = V D^{n} V^{-1}

If P has n linearly independent eigenvectors $v_1, \ldots, v_n$ with eigenvalues $\lambda_1, \ldots, \lambda_n$, put them as columns of V and the eigenvalues on the diagonal of D. Then $P = V D V^{-1}$. Since the inner $V^{-1}V$ cancel, $P^n = V D^n V^{-1}$, and D^n is easy because we only raise each diagonal entry to the power n. Distinct eigenvalues guarantee that P is diagonalisable, which is why we arranged the matrix so that all eigenvalues are real and distinct.

Write the starting vector as $x_0 = c_1v_1 + c_2v_2 + \ldots + c_nv_n$, where $c = V^{-1}x_0$. Then

$$x_k = c_1(1)^k v_1 + c_2\lambda_2^k v_2 + c_3\lambda_3^k v_3 + \ldots$$

The terms with $|\lambda_j| < 1$ die out as k grows, so only c_1v_1 survives, and that is the steady state. The speed of convergence is controlled by the largest of the remaining $|\lambda_j|$, namely $|\lambda_2|$: the closer it is to 1, the slower the convergence, and the distance to the steady state shrinks roughly like $|\lambda_2|^k$.

2.5 A worked 2 × 2 example

Let T = [[0.9, 0.2],[0.1, 0.8]] (two cities; 10% of city 1 move to city 2 each year and 20% of city 2 move to city 1). Columns sum to 1.

- **Eigenvalues:** the characteristic equation is $\lambda^2 - (\text{trace})\lambda + \det = \lambda^2 - 1.7\lambda + 0.70 = 0$, so λ = 1 and λ = 0.7.
- **λ = 1:** $(T - I)v = 0$ gives $-0.1a + 0.2b = 0$, so $a = 2b$ and $v_1 = (2, 1)$. Normalising to sum 1 gives the steady state $\pi = (2/3, 1/3)$.
- **λ = 0.7:** $(T - 0.7I)v = 0$ gives $0.2a + 0.2b = 0$, so $v_2 = (1, -1)$ (entries sum to 0, as predicted).
- **Diagonalisation:** V = [[2, 1],[1, −1]], D = diag(1, 0.7). Hence $T^n = V D^n V^{-1} \to$ [[2/3, 2/3],[1/3, 1/3]] as $n \to \infty$ because $0.7^n \to 0$.
- **Check:** starting from (0.5, 0.5) the code gives (0.5000, 0.5000) → (0.5500, 0.4500) → (0.5850, 0.4150) → (0.6095, 0.3905), moving towards (0.6667, 0.3333).

3 Data and Assumptions

3.1 Regions and initial population

We use four regions at district level. The initial populations in Table 1 are **approximate figures from the Census of India 2011** and are **to be verified by the group** against the official tables. The total is 21,786,473. All code inputs are in one editable dictionary at the top of the script, so correcting a figure only needs one edit and a re-run.

Table 1: Approximate initial populations (Census 2011, to be verified)

Region	Unit used	Population (approx.)	Initial share
Delhi	NCT of Delhi	16,787,941	77.06%
Gurugram	Gurgaon district	1,514,085	6.95%
Noida	Gautam Buddha Nagar district	1,674,714	7.69%
Faridabad	Faridabad district	1,809,733	8.31%

3.2 Simulated transition probabilities

The transition probabilities in this project are simulated. Public data on yearly movement between individual NCR cities, in the form of a complete origin–destination matrix, are not available at this granularity (Census migration tables are collected only occasionally, by state or district, and use different time windows). We therefore designed P by hand using plain-language reasoning about how the four places behave, and we kept it within the rules we set ourselves: retention between 0.88 and 0.96, all entries strictly positive, columns summing to 1, and real distinct eigenvalues. Table 2 explains each column.

Table 2: Justification of each column of P (simulated)

Column (from)	Stay	Where the movers go	Plain-language reasoning
Delhi	0.950	Gurugram 0.020, Noida 0.020, Faridabad 0.010	Highest retention: the largest job market, established housing and family ties. Movers prefer Gurugram and Noida (IT and corporate jobs) over Faridabad.
Gurugram	0.890	Delhi 0.060, Noida 0.030, Faridabad 0.020	Many workers are recent arrivals with weaker roots; some return to Delhi for housing cost, family or education. Few move to Faridabad.
Noida	0.890	Delhi 0.060, Gurugram 0.030, Faridabad 0.020	Similar to Gurugram (IT and job-driven residents). Slightly more movement back to Delhi, which is adjacent.
Faridabad	0.880	Delhi 0.070, Gurugram 0.030, Noida 0.020	Lowest retention: some residents move to Delhi (closest large job market) and Gurugram (jobs along the same corridor), fewer to Noida.

3.3 Assumptions

- Time step is one year and P is constant in time (the same every year).
- The system is closed: only the four regions exist; no births, deaths, or movement to or from the rest of India.
- The process is memoryless: where a person goes next depends only on where they live now.
- Each person in a region has the same probabilities (no age, income or occupation differences).
- Populations are approximate Census 2011 figures at district level; the year 2011 is only the starting point of the simulation and we do not claim real forecasts.
- In the sensitivity study, '±2%' means each entry of P is multiplied by a random factor in [0.98, 1.02] and each column is then renormalised to sum to 1.

4 Model Construction

The final transition matrix (rows = region next year, columns = region this year) is shown in Table 3 and as a heat map in Figure 1.

Table 3: Final transition matrix P (simulated), $x_{k+1} = P\,x_k$

to \ from	Delhi	Gurugram	Noida	Faridabad
Delhi	0.950	0.060	0.060	0.070
Gurugram	0.020	0.890	0.030	0.030
Noida	0.020	0.030	0.890	0.020
Faridabad	0.010	0.020	0.020	0.880

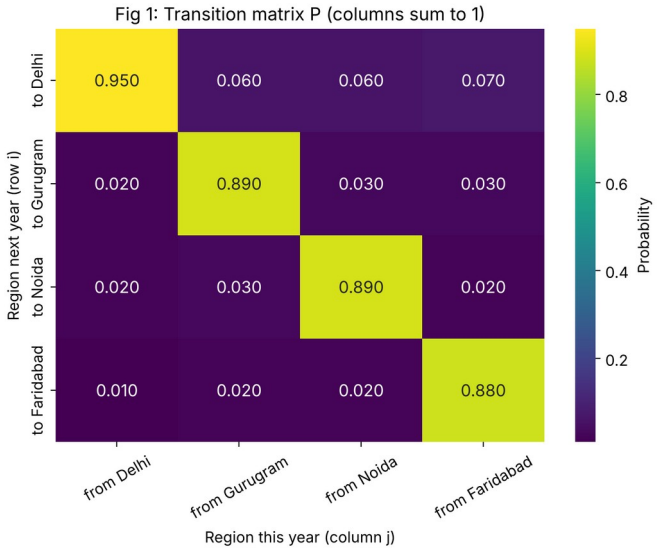


Figure 1: Heat map of the transition matrix P. Large diagonal values show that most people stay where they are.

Fig 2: Yearly movement between regions (arrow thickness ~ probability; colour = origin)

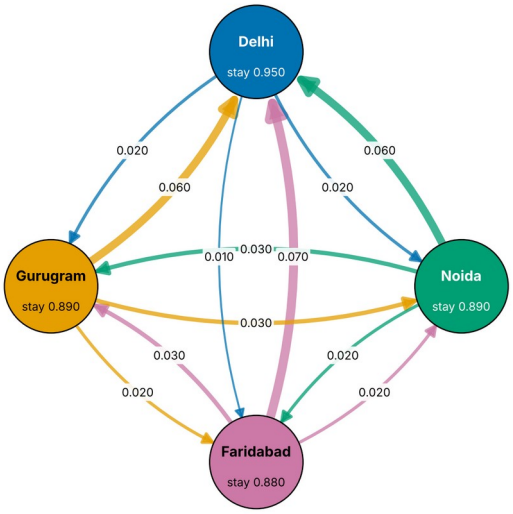


Figure 2: Flow diagram of the four regions. Arrow thickness is proportional to the transition probability.

4.1 Verification of properties

Table 4: Properties of P checked by the program (all assertions passed)

Property	Result
Every column sums to 1	Yes (column sums = 1.000, 1.000, 1.000, 1.000)
All entries non-negative	Yes
Retention (diagonal) between 0.88 and 0.96	Yes (0.950, 0.890, 0.890, 0.880)
Regular: smallest k with $P^k > 0$	k = 1 (already P itself is strictly positive)
Eigenvalues real and distinct	Yes (1.0000, 0.8862, 0.8638, 0.8600)
No region below 5% in steady state	Yes (smallest share 10.32%)

5 Results

5.1 Population projections

Starting from the approximate 2011 populations x_0, we computed $x_{k+1} = P\,x_k$ repeatedly for 30 years. Table 5 gives the population counts and Table 6 the shares for selected years.

Table 5: Projected population (persons) after k years

k	Delhi	Gurugram	Noida	Faridabad
0	16,787,941	1,514,085	1,674,714	1,809,733
1	16,266,553	1,787,828	1,907,871	1,824,220
2	15,802,663	2,028,461	2,113,456	1,841,893
5	14,696,386	2,589,839	2,595,438	1,904,811
10	13,532,662	3,157,162	3,085,853	2,010,797
20	12,528,941	3,618,544	3,485,929	2,153,059
30	12,221,876	3,750,675	3,599,354	2,214,569

Table 6: Projected share of total population after k years

k	Delhi	Gurugram	Noida	Faridabad
0	77.06%	6.95%	7.69%	8.31%
1	74.66%	8.21%	8.76%	8.37%
2	72.53%	9.31%	9.70%	8.45%
5	67.46%	11.89%	11.91%	8.74%
10	62.11%	14.49%	14.16%	9.23%
20	57.51%	16.61%	16.00%	9.88%
30	56.10%	17.22%	16.52%	10.16%

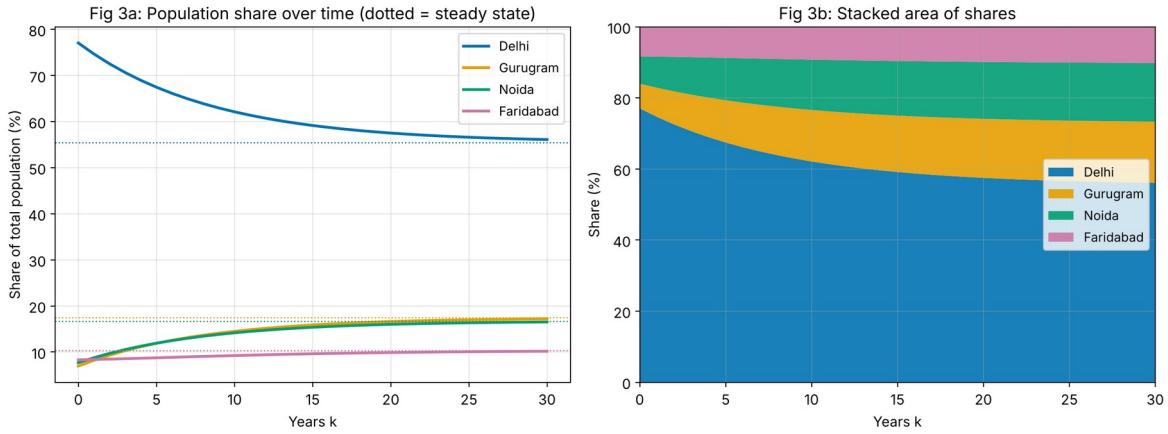


Figure 3: Population shares over 30 years (left: lines with dotted steady-state levels; right: stacked area).

The total stays at 21,786,473 in every year because P is column-stochastic. Delhi's share falls from 77.1% to 56.1% after 30 years, while Gurugram, Noida and Faridabad grow. A hand calculation for one entry is shown in Appendix B.

5.2 Matrix powers

Instead of repeating the step, we can compute P^k directly. Table 7 shows P^{10} and P^{50}. In P^{10} the columns are still different, but in P^{50} each row is nearly constant: all four columns have become almost the same vector. The largest gap between columns in any row is 0.310 for P^{10} and 0.0027 for P^{50}. That common column is the steady state.

Table 7: Selected matrix powers

Power	Delhi col	Gurugram col	Noida col	Faridabad col
P^{10} Delhi	0.6898	0.3796	0.3796	0.4097
Gurugram	0.1227	0.3742	0.1529	0.1514
Noida	0.1203	0.1490	0.3703	0.1218
Faridabad	0.0671	0.0972	0.0972	0.3171
P^{50} Delhi	0.5559	0.5533	0.5533	0.5540
Gurugram	0.1742	0.1755	0.1750	0.1748
Noida	0.1670	0.1676	0.1681	0.1671
Faridabad	0.1028	0.1036	0.1036	0.1040

5.3 Steady state: three independent methods

- **(a) Eigenvector for λ = 1,** normalised so its entries sum to 1.
- **(b) Linear system.** We solve $(P - I)\pi = 0$ together with $\pi_1+\pi_2+\pi_3+\pi_4 = 1$ by Gaussian elimination: the last row of P − I is replaced by a row of ones and the right-hand side by (0, 0, 0, 1).
- **(c) High power.** Every column of P^{200} is the steady state.

Table 8: Steady state from the three methods

Method	Delhi	Gurugram	Noida	Faridabad
(a) Eigenvector	0.55483871	0.17465438	0.16728111	0.10322581
(b) Linear system	0.55483871	0.17465438	0.16728111	0.10322581
(c) Column of P^{200}	0.55483871	0.17465438	0.16728111	0.10322581
Steady state (%)	55.48%	17.47%	16.73%	10.32%

The largest difference between the methods is 1.52e-11, far below the tolerance 10^{-8} that we required. The steady-state counts are Delhi 12,087,979, Gurugram 3,805,103, Noida 3,644,465, Faridabad 2,248,926. As a check of the balance idea, at the steady state the share leaving Delhi each year (0.05 × 0.5548 = 0.0277) equals the share arriving in Delhi (0.06×0.1747 + 0.06×0.1673 + 0.07×0.1032 = 0.0277).

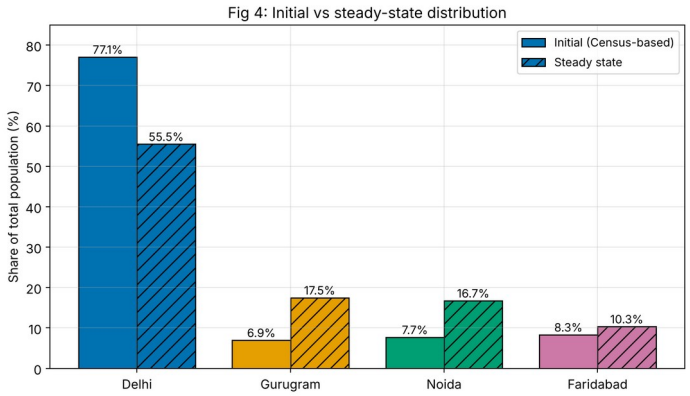


Figure 4: Initial (Census-based, approximate) and steady-state shares.

5.4 Eigen-analysis and diagonalisation

The eigenvalues of P are all real and distinct: $\lambda_1 = 1.000000$, $\lambda_2 = 0.886180$, $\lambda_3 = 0.863820$, $\lambda_4 = 0.860000$ (Figure 9). Hence P is diagonalisable: $P = V D V^{-1}$ with $D = \text{diag}(\lambda_1, \ldots, \lambda_4)$ and V holding the eigenvectors (Table 9). The reconstruction error $\max|V D V^{-1} - P|$ is 4.4e-16. For n = 10, the matrix $V D^{10} V^{-1}$ agrees with the direct power P^{10} to within 5.0e-15.

Table 9: Eigenvector matrix V (first column normalised to sum 1; the others to length 1)

	v_1 (λ=1)	v_2	v_3	v_4
Delhi	0.5548	0.8615	0.1750	0.0000
Gurugram	0.1747	-0.3290	-0.4582	0.7071
Noida	0.1673	-0.2034	0.7413	-0.7071
Faridabad	0.1032	-0.3290	-0.4582	0.0000

Writing the starting share vector as $x_0 = c_1v_1 + c_2v_2 + c_3v_3 + c_4v_4$ gives $c_1 = 1.0000$, $c_2 = 0.2827$, $c_3 = -0.1591$, $c_4 = -0.1202$. The coefficient c_1 equals 1 because the other eigenvectors sum to 0 and x_0 sums to 1. After 30 years the factors λ_j^{30} are 1.00e+0, 2.66e-2, 1.24e-2, 1.08e-2: the λ = 1 component stays at full strength, while the others have shrunk to about 1–3% of their start. This is the algebraic reason for convergence.

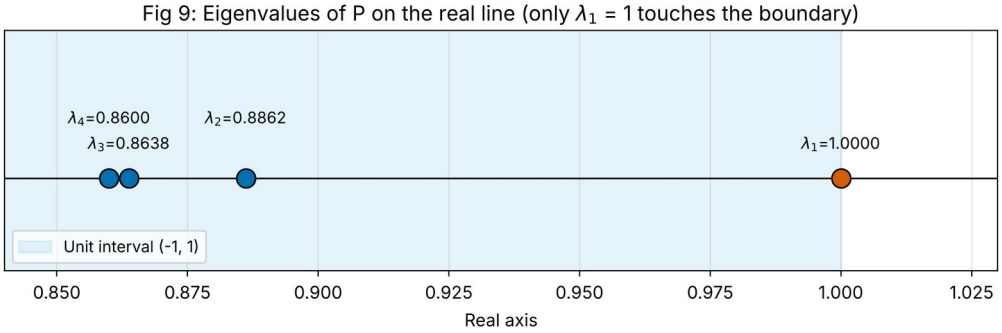


Figure 9: Eigenvalues of P on the real line. Only λ1 = 1 lies on the boundary of the unit interval.

5.5 Convergence speed

We measure the distance to the steady state by $d_k = \Sigma|x_{k,i} - \pi_i|$ using shares. Because $d_k \approx C|\lambda_2|^k$, taking logarithms gives the estimate $k \approx \ln(\varepsilon/C)/\ln|\lambda_2|$ for the years needed to reach distance ε. With $|\lambda_2|$ =

0.8862 and a fitted constant C = 0.4537 we get Table 10. The slowest mode loses half of its size every 5.7 years.

Table 10: Years until the distance to the steady state drops below a threshold

Threshold	Actual (simulation)	Theory from $\|\lambda_2\|^k$
1% ($d_k < 0.01$)	32	32
0.1% ($d_k < 0.001$)	51	51

Fig 5: Convergence to steady state (log scale)

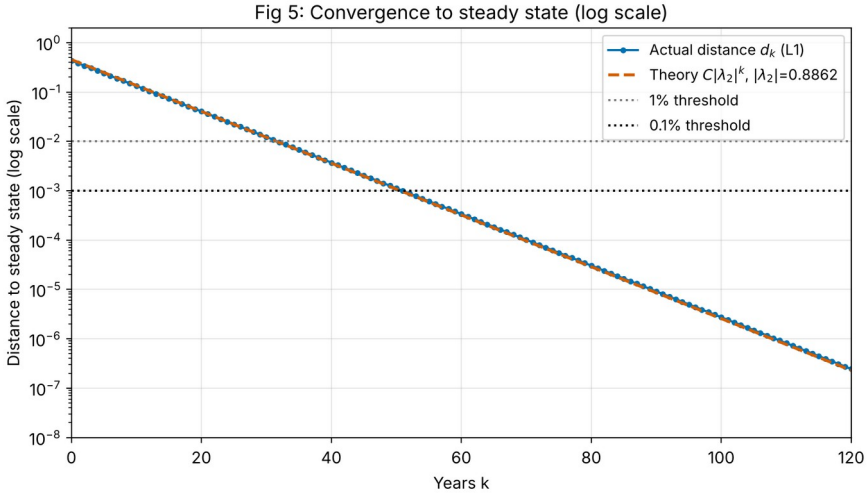


Figure 5: Distance to the steady state on a log scale. The straight line shows geometric decay at rate $|\lambda_2|$.

Different starting points. Figure 6 starts the system from four very different distributions (the Census-based one, everyone in Delhi, uniform, everyone in Faridabad). All of them end at the same steady state, as the theorem predicts: the long-term result depends on P, not on the starting point.

Fig 6: Different starting points converge to the same steady state

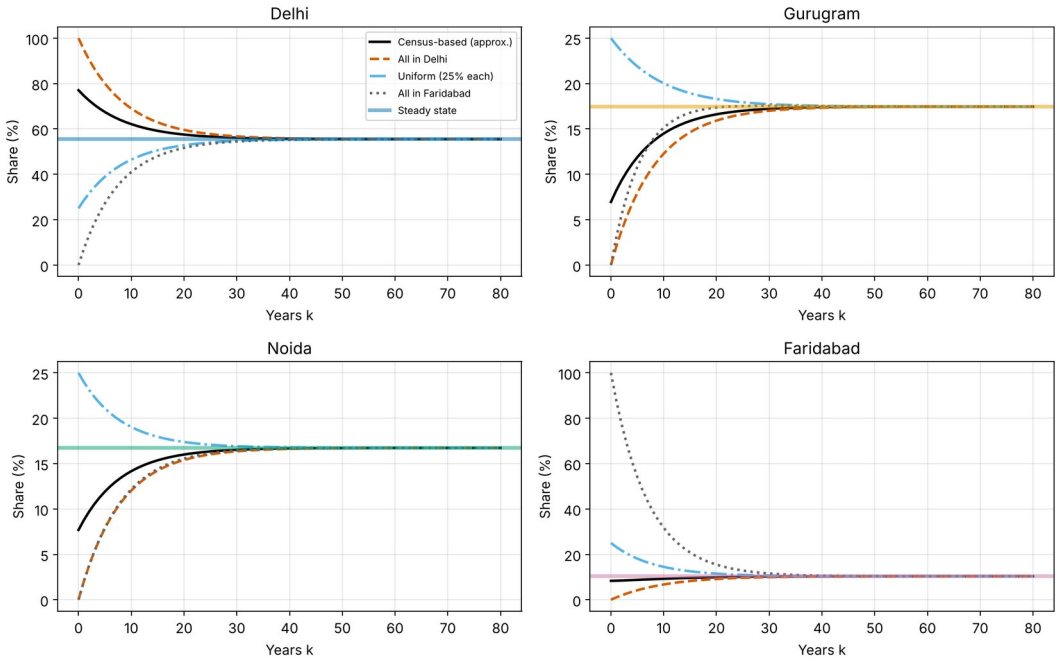


Figure 6: Four different starting distributions converge to the same steady state.

6 Scenario and Sensitivity Analysis

6.1 Scenarios (simulated what-if changes)

- **Scenario A – Faridabad improves infrastructure.** Faridabad's retention rises from 0.880 to 0.920, and the flows into Faridabad increase (from Delhi 0.010 → 0.015, from Gurugram 0.020 → 0.030, from Noida 0.020 → 0.030). The other columns are rebalanced so that every column still sums to 1.
- **Scenario B – Job boom in Gurugram and Noida.** Delhi's movers to Gurugram and Noida rise from 0.020 and 0.020 to 0.035 each (Delhi's retention drops to 0.920), and Gurugram and Noida each retain more people (0.920). Columns are rebalanced.

Table 11: Steady-state shares and $|\lambda_2|$ under each scenario

Scenario	Delhi	Gurugram	Noida	Faridabad	$\|\lambda_2\|$
Baseline	55.48%	17.47%	16.73%	10.32%	0.8862
A: Faridabad	46.62%	16.60%	15.86%	20.91%	0.9015
B: Job boom	34.15%	28.47%	27.53%	9.85%	0.8950
Change A (pp)	-8.86	-0.86	-0.87	+10.59	0.0153
Change B (pp)	-21.33	+11.00	+10.80	-0.48	0.0088

Fig 7: Steady state under scenarios (hatch pattern = scenario)

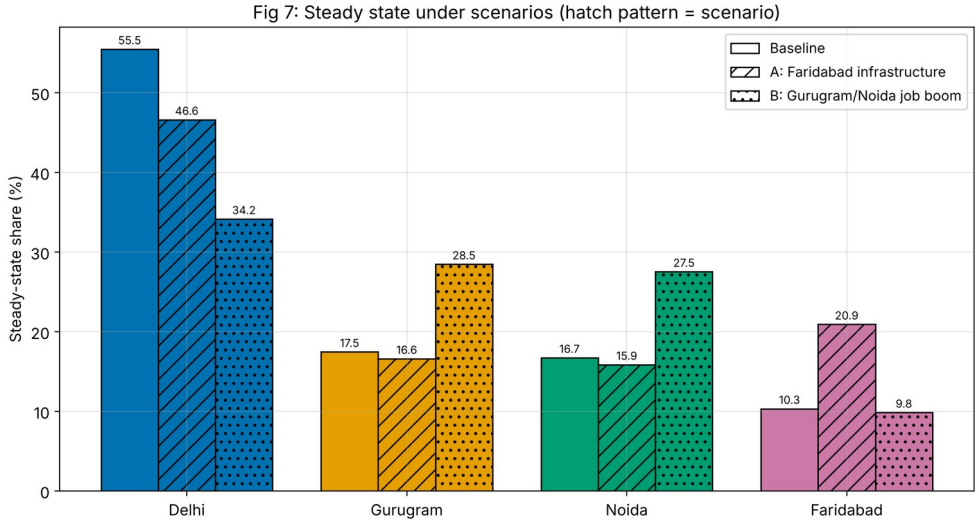


Figure 7: Steady-state shares under the baseline and the two scenarios.

In scenario A Faridabad's long-term share rises from 10.3% to 20.9%, mostly at the expense of Delhi (55.5% → 46.6%). In scenario B Gurugram and Noida jump to 28.5% and 27.5%, while Delhi falls to 34.2%. In both scenarios $|\lambda_2|$ rises slightly (0.8862 → 0.9015 and 0.8950), so the new steady state would be approached a little more slowly. Small changes in yearly probabilities therefore produce large changes in the long-run shares.

6.2 Sensitivity analysis

To test how much our conclusions depend on the exact numbers in P, we ran 1000 random experiments. In each run every entry of P was multiplied by a random factor between 0.98 and 1.02, the columns were renormalised, and the steady state was recomputed (random seed 42).

Table 12: Steady-state share (%) over 1000 perturbed matrices

Region	Baseline	Mean	Std. dev.	Min	Max
Delhi	55.48	55.47	0.39	54.36	56.68
Gurugram	17.47	17.48	0.24	16.74	18.20
Noida	16.73	16.73	0.23	15.94	17.49
Faridabad	10.32	10.33	0.14	9.95	10.74

Fig 8: Sensitivity of steady-state shares to +/-2% perturbations (1000 runs)

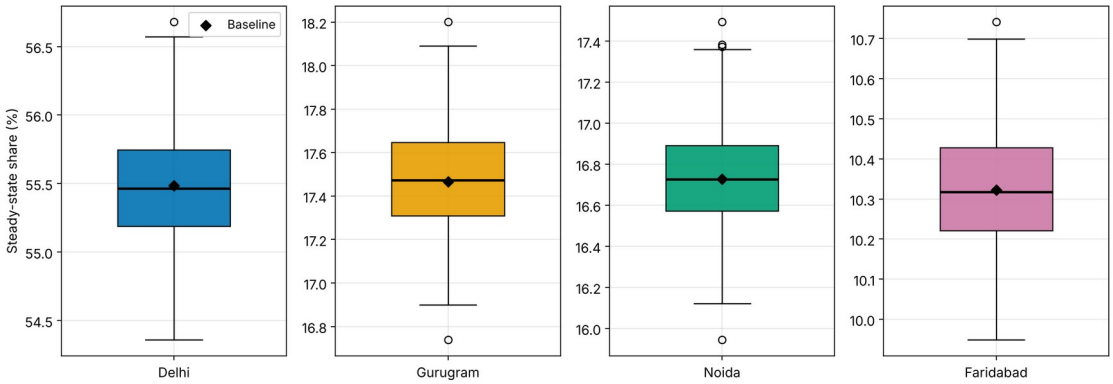


Figure 8: Distribution of the steady-state shares over 1000 perturbations (diamond = baseline).

Conclusion on robustness. The standard deviation of each share is at most 0.39 percentage points, and the largest deviation from the baseline over all runs is 1.20 percentage points. $|\lambda_2|$ stays between 0.8835 and 0.8894. So the qualitative picture (Delhi largest, Faridabad smallest, convergence in several decades) is robust to ±2% errors in P. It is, however, not robust to the larger structural changes of the scenarios, which is why accurate data on retention would matter.

7 Discussion

7.1 What the numbers mean in plain English

Every year about 5% of Delhi's residents move to the other three regions, but Delhi receives 6–7% of the residents of each of the others. Because Delhi starts with 77% of the people, the movers going into Delhi are few in absolute terms compared with the many movers going out, so Delhi's share falls steadily until outflow equals inflow at 55.5%. The other three regions grow towards 17.5%, 16.7% and 10.3%. After about 32 years the distribution is within 1% (in total distance) of the steady state, so the long-term picture is reached within a generation or two.

We must be careful: in this model the *number* of people living in Delhi falls from 16,787,941 to 12,221,876 after 30 years. Real Delhi has not behaved this way, because births and migration from other states more than compensate. This is an artefact of our closed-system assumption and simulated probabilities, not a prediction, and we do not claim it as one.

7.2 Limitations

- **Closed system:** no births, deaths, or movement into or out of the four regions.
- **Constant P:** real movement probabilities change with the economy, policy and infrastructure.
- **Memoryless:** a person's next move is assumed to depend only on where they live now, not on how long they have lived there.
- **Simulated data:** the transition probabilities are not measured; our conclusions are about the model, not about reality.

- **Constant retention and identical people:** all residents of a region are treated the same.
- **Approximate populations:** Census 2011 figures at district level, which are now old.

7.3 What if P is not regular?

The steady-state theorem needs regularity. If a region has retention 1 (an absorbing state), people who enter it never leave. In our 3-state example with such a state, starting from state 1 the population after 200 steps is (0.000003, 0.000004, 0.999993): everybody ends in the absorbing state, and the steady state is not driven by inflow–outflow balance. A periodic chain such as P = [[0, 1],[1, 0]] has eigenvalues 1 and −1; starting from (1, 0) the vector flips between (1, 0) and (0, 1) forever and has no limit. For these chains the $|\lambda_2| < 1$ argument fails, so the result of Section 2.3 cannot be used. Our matrix is strictly positive, so it is regular.

7.4 What real data would change

With real data (for example yearly origin–destination counts from administrative or survey sources), the diagonal entries would probably be much closer to 1 and the off-diagonal entries smaller and less symmetric. We would also be able to add rows and columns for 'rest of India', births and deaths (making the matrix non-stochastic or adding an input vector), and let P change with time. The mathematical tools (matrix powers, eigenvalues, steady state) would remain the same.

8 Conclusion

We built a regular, column-stochastic transition matrix for four NCR regions, projected the population by matrix multiplication and found the steady state (Delhi 55.48%, Gurugram 17.47%, Noida 16.73%, Faridabad 10.32%) by three independent methods that agree. Diagonalisation shows that the $\lambda = 1$ eigenvector survives while the other components decay at the rates 0.8862^k, 0.8638^k and 0.8600^k, and $|\lambda_2|$ predicts the convergence time well. The scenarios show that small changes in yearly probabilities shift the long-run shares considerably, whereas ±2% random errors change them by at most 1.2 percentage points. The main limitation is that the transition probabilities are simulated; the project demonstrates the method, and real migration data would be needed for real predictions.

9 References

1. D. Poole, Linear Algebra: A Modern Introduction (course textbook; chapter on Markov chains).
2. D. G. Zill, Advanced Engineering Mathematics (for matrices and eigenvalue methods).
3. Census of India 2011, Office of the Registrar General & Census Commissioner, India (general source of the approximate population figures; to be verified by the group).
4. Python 3, NumPy, Matplotlib, pandas and seaborn software documentation (used for all computations and figures).

Individual contribution (placeholder – edit before submission)

Member	Roll No.	Contribution (edit)
Mansi Awasthi	250450	e.g. Mathematical background and 2 × 2 example
Prince Kumar Yadav	250500	e.g. Python code, figures and results
Shobhit Rai	250557	e.g. Scenario and sensitivity analysis
Ishank Chauhan	250406	e.g. Report editing, data and coordination

Appendix A: Full Code Listing (project_code.py)

```
#!/usr/bin/env python3
"""
Group 14 - MTH2802 Linear Algebra & Differential Equations (BML Munjal University)
Project : Predicting Population Movement Using Transition Matrices
Convention (Poole): P is COLUMN-stochastic, x_{k+1} = P x_k,
                    P[i][j] = Prob(person in region j this year lives in region i next year).
Run     : python project_code.py      (creates figures/, results.json, results.txt next to this file)
NOTE    : Transition probabilities are SIMULATED. Populations are APPROXIMATE (Census 2011).
"""
import json, math, warnings
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.patches import FancyArrowPatch
try:
    import seaborn as sns
except ImportError:           # graceful fallback: heatmap drawn with matplotlib only
    sns = None
warnings.filterwarnings("ignore")
np.random.seed(42)                          # reproducibility
rng = np.random.default_rng(42)

OUT = Path(__file__).resolve().parent
FIG = OUT / "figures"; FIG.mkdir(exist_ok=True)
DPI = 200

# ------------------------------------------------------------------------------
# 0. EDITABLE INPUTS
# ------------------------------------------------------------------------------
# APPROXIMATE populations, from Census of India 2011 (district level), TO BE VERIFIED BY THE GROUP.
INITIAL_POPULATION = {
    "Delhi":     16_787_941,   # NCT of Delhi (approx., Census 2011)
    "Gurugram":   1_514_085,   # Gurgaon district (approx., Census 2011)
    "Noida":      1_674_714,   # Gautam Buddha Nagar district (approx., Census 2011)
    "Faridabad":  1_809_733,   # Faridabad district (approx., Census 2011)
}
REGIONS = list(INITIAL_POPULATION)
n = len(REGIONS)
COLORS = {"Delhi": "#0072B2", "Gurugram": "#E69F00", "Noida": "#009E73", "Faridabad": "#CC79A7"}  # Okabe-Ito
(colour-blind safe)
REG_COL = [COLORS[r] for r in REGIONS]

# SIMULATED transition matrix (column j = where people living in region j go next year). Columns sum to 1.
#                 from: Delhi  Gurugram Noida  Faridabad
P = np.array([[0.950, 0.060, 0.060, 0.070],    # to Delhi
              [0.020, 0.890, 0.030, 0.030],    # to Gurugram
              [0.020, 0.030, 0.890, 0.020],    # to Noida
              [0.010, 0.020, 0.020, 0.880]])   # to Faridabad

R = {}                      # everything that goes in results.json
LOG = []                    # lines for results.txt
PASSED = []                 # assertion log
def out(s=""):
    print(s); LOG.append(str(s))
def check(cond, msg):
    assert cond, "FAILED: " + msg
    PASSED.append(msg); out(f"[PASS] {msg}")
def L(a, d=6): return np.round(np.asarray(a, dtype=float), d).tolist()
def savefig(name):
    plt.tight_layout(); plt.savefig(FIG / name, dpi=DPI, bbox_inches="tight"); plt.close()
def pct(v): return 100 * np.asarray(v)

plt.rcParams.update({"font.size": 11, "axes.titlesize": 13, "axes.labelsize": 11, "legend.fontsize": 10,
                     "axes.grid": True, "grid.alpha": 0.3})

def steady_eig(M):
```

```
    """Steady state via eigenvector for lambda = 1, normalised to sum 1."""
    w, V = np.linalg.eig(M)
    i = np.argmin(abs(w - 1)); v = np.real(V[:, i]); return v / v.sum()

def gauss_solve(A, b, verbose=False):
    """Gaussian elimination with partial pivoting + back substitution (course-style)."""
    A = A.astype(float).copy(); b = b.astype(float).copy(); m = len(b)
    for c in range(m):
        p = c + np.argmax(abs(A[c:, c])); A[[c, p]] = A[[p, c]]; b[[c, p]] = b[[p, c]]
        for r in range(c + 1, m):
            f = A[r, c] / A[c, c]; A[r, c:] -= f * A[c, c:]; b[r] -= f * b[c]
    if verbose:
        out("Row-echelon form [A | b] after forward elimination:")
        for r in range(m): out("   " + "  ".join(f"{A[r, j]:9.5f}" for j in range(m)) + "  |  " +
f"{b[r]:9.5f}")
    x = np.zeros(m)
    for r in range(m - 1, -1, -1): x[r] = (b[r] - A[r, r + 1:] @ x[r + 1:]) / A[r, r]
    return x

def second_eig(M):
    ev = np.linalg.eigvals(M); ev = ev[np.argsort(-abs(ev))]; return abs(ev[1])

# ------------------------------------------------------------------------------
# 1. BUILD P AND VERIFY BASIC PROPERTIES
# ------------------------------------------------------------------------------
out("=" * 78); out("1. TRANSITION MATRIX P (SIMULATED, column-stochastic)"); out("=" * 78)
out(pd.DataFrame(P, index=["to " + r for r in REGIONS], columns=["from " + r for r in REGIONS]).to_string())
check(np.allclose(P.sum(axis=0), 1, atol=1e-12), "Every column of P sums to 1")
check(all(round(float(c), 3) == 1.0 for c in P.sum(axis=0)), "Column sums equal 1 when rounded to 3 decimals")
check((P >= 0).all(), "All entries of P are non-negative")
check((P > 0).all(), "All entries of P strictly positive (so P^1 > 0)")
check(all(0.88 <= P[i, i] <= 0.96 for i in range(n)), "Retention (diagonal) lies in [0.88, 0.96]")
k_reg = next(k for k in range(1, 50) if (np.linalg.matrix_power(P, k) > 0).all())
check(k_reg == 1, f"Regularity: smallest k with P^k > 0 is k = {k_reg}")
R["P"] = L(P, 3); R["regions"] = REGIONS; R["retention"] = {r: float(P[i, i]) for i, r in enumerate(REGIONS)}
R["column_sums"] = L(P.sum(axis=0), 12); R["smallest_k_regular"] = k_reg
R["initial_population_approx_census2011"] = INITIAL_POPULATION

x0 = np.array(list(INITIAL_POPULATION.values()), dtype=float); N0 = x0.sum()
s0 = x0 / N0
R["total_population"] = int(N0); R["initial_share"] = L(s0)
out(f"\nTotal initial population (approx.) = {int(N0):,};  initial shares = {np.round(s0, 4)}")

# ------------------------------------------------------------------------------
# 9a. HAND-CHECKABLE EXAMPLE: x_1 for Gurugram, step by step
# ------------------------------------------------------------------------------
out("\n" + "=" * 78); out("HAND-CHECKABLE EXAMPLE: population of Gurugram after 1 year"); out("=" * 78)
row = 1
terms = [P[row, j] * x0[j] for j in range(n)]
out("x_1[Gurugram] = sum_j P[Gurugram][j] * x_0[j]")
for j in range(n): out(f"   from {REGIONS[j]:<10}: {P[row, j]:.3f} x {x0[j]:>12,.0f} = {terms[j]:>14,.1f}")
out(f"   TOTAL = {sum(terms):,.1f}  (people in Gurugram after 1 year)")
R["hand_example"] = {"region": REGIONS[row], "row_of_P": L(P[row], 3), "x0": x0.tolist(),
                     "terms": L(terms, 1), "x1_value": round(float(sum(terms)), 1)}
x1_check = P @ x0
check(abs(x1_check[row] - sum(terms)) < 1e-6, "Hand-calculation of x_1[Gurugram] matches P @ x_0")
check(abs(x1_check.sum() - N0) < 1e-6, "Total population conserved after 1 step (closed system)")

# ------------------------------------------------------------------------------
# 2. PROJECTIONS x_0 ... x_30 AND MATRIX POWERS
# ------------------------------------------------------------------------------
out("\n" + "=" * 78); out("2. PROJECTIONS x_{k+1} = P x_k"); out("=" * 78)
K = 30
X = np.zeros((K + 1, n)); X[0] = x0
for k in range(K): X[k + 1] = P @ X[k]
S = X / X.sum(axis=1, keepdims=True)
show = [0, 1, 2, 5, 10, 20, 30]
tab = pd.DataFrame({"k (years)": show})
for i, r in enumerate(REGIONS): tab[r + " (persons)"] = [round(X[k, i]) for k in show]
out(tab.to_string(index=False))
tab2 = pd.DataFrame({"k (years)": show})
```

```
for i, r in enumerate(REGIONS): tab2[r + " (%)"] = [round(100 * S[k, i], 2) for k in show]
out("\nShares in percent:"); out(tab2.to_string(index=False))
R["projection_k"] = show
R["projection_counts"] = {str(k): [round(float(v)) for v in X[k]] for k in show}
R["projection_shares"] = {str(k): L(S[k], 6) for k in show}
check(np.allclose(X.sum(axis=1), N0), "Total population constant for all k = 0..30")
check(np.allclose(S.sum(axis=1), 1), "Shares sum to 1 for all k")

out("\nMATRIX POWERS (columns converge to the same vector):")
R["matrix_powers"] = {}; R["power_column_spread"] = {}
for p_ in [2, 5, 10, 50]:
    Pp = np.linalg.matrix_power(P, p_)
    out(f"\nP^{p_} =\n{np.round(Pp, 4)}")
    spread = float(np.max(Pp.max(axis=1) - Pp.min(axis=1)))
    out(f"   largest difference between columns in any row = {spread:.2e}")
    R["matrix_powers"][f"P^{p_}"] = L(Pp, 6); R["power_column_spread"][f"P^{p_}"] = spread
check(R["power_column_spread"]["P^50"] < 1e-2, "Columns of P^50 are (almost) identical")

# ------------------------------------------------------------------------------
# 3. STEADY STATE - THREE INDEPENDENT METHODS
# ------------------------------------------------------------------------------
out("\n" + "=" * 78); out("3. STEADY STATE (three methods)"); out("=" * 78)
pi_a = steady_eig(P)
A = P - np.eye(n); A_c = A.copy(); A_c[-1, :] = 1.0; b_c = np.zeros(n); b_c[-1] = 1.0
out("Method (b): replace the last equation of (P-I)pi = 0 by pi1+pi2+pi3+pi4 = 1.")
out("Augmented matrix [A | b] BEFORE elimination:")
for r in range(n): out("   " + "  ".join(f"{A_c[r, j]:9.4f}" for j in range(n)) + "  |  " + f"{b_c[r]:6.1f}")
pi_b = gauss_solve(A_c, b_c, verbose=True)
pi_c = np.linalg.matrix_power(P, 200)[:, 0]
for nm, v in zip(["(a) eigenvector", "(b) linear system", "(c) P^200 column"], [pi_a, pi_b, pi_c]):
    out(f"  {nm:<20}: " + "  ".join(f"{REGIONS[i]} {v[i]:.8f}" for i in range(n)))
check(np.max(abs(pi_a - pi_b)) < 1e-8, "Steady state: eigenvector method == linear-system method (<1e-8)")
check(np.max(abs(pi_a - pi_c)) < 1e-8, "Steady state: eigenvector method == P^200 method (<1e-8)")
check(np.max(abs(pi_b - pi_c)) < 1e-8, "Steady state: linear-system method == P^200 method (<1e-8)")
pi = pi_a
check(np.allclose(P @ pi, pi, atol=1e-12), "P pi = pi holds")
check(pi.min() > 0.05, f"No region collapses below 5% (smallest share = {100 * pi.min():.2f}%)")
R["steady_state"] = {"eigenvector": L(pi_a, 10), "linear_system": L(pi_b, 10), "P200": L(pi_c, 10),
                     "max_abs_difference": float(max(abs(pi_a - pi_b).max(), abs(pi_a - pi_c).max()))}
R["steady_state_counts"] = [round(float(v)) for v in pi * N0]

# ------------------------------------------------------------------------------
# 4. EIGEN-DECOMPOSITION
# ------------------------------------------------------------------------------
out("\n" + "=" * 78); out("4. EIGEN-DECOMPOSITION  P = V D V^-1"); out("=" * 78)
w, V = np.linalg.eig(P)
check(np.allclose(w.imag, 0), "All eigenvalues are real")
w = w.real; V = V.real
order = np.argsort(-w); w = w[order]; V = V[:, order]
check(len(set(np.round(w, 8))) == n, "All eigenvalues are distinct")
check(np.min(np.diff(np.sort(w))) > 1e-3, "Eigenvalues separated by more than 1e-3")
for j in range(n):                        # normalisation: v1 sums to 1 (= steady state); others unit length,
first nonzero entry > 0
    if j == 0: V[:, j] = V[:, j] / V[:, j].sum()
    else:
        V[:, j] = V[:, j] / np.linalg.norm(V[:, j])
        if V[np.argmax(abs(V[:, j]) > 1e-12), j] < 0: V[:, j] *= -1
D = np.diag(w); Vinv = np.linalg.inv(V)
rec_err = np.max(abs(V @ D @ Vinv - P))
out("Eigenvalues: " + ", ".join(f"{x:.6f}" for x in w))
out("Eigenvectors (columns of V):\n" + str(np.round(V, 5)))
out(f"Reconstruction error max|V D V^-1 - P| = {rec_err:.2e}")
check(rec_err < 1e-12, "P = V D V^-1 reconstruction error < 1e-12")
check(abs(w[0] - 1) < 1e-12, "Largest eigenvalue is exactly 1 (lambda_1 = 1)")
check(abs(V[:, 0] - pi).max() < 1e-10, "Eigenvector for lambda = 1 equals the steady state")
check(all(abs(V[:, j].sum()) < 1e-10 for j in range(1, n)), "Other eigenvectors have components summing to 0")
P10_dir = np.linalg.matrix_power(P, 10); P10_diag = V @ np.diag(w ** 10) @ Vinv
err10 = np.max(abs(P10_dir - P10_diag)); out(f"P^10 via V D^10 V^-1 vs matrix_power: max error = {err10:.2e}")
check(err10 < 1e-12, "P^10 = V D^10 V^-1 matches np.linalg.matrix_power(P,10)")
c = Vinv @ s0                                  # coefficients of x_0 (shares)
out("\nDecomposition of the initial SHARE vector: x_0 = c1 v1 + c2 v2 + c3 v3 + c4 v4")
```

```
for j in range(n): out(f"   c{j + 1} = {c[j]: .6f}   (lambda_{j + 1} = {w[j]:.6f}, lambda^30 = {w[j] **
30:.3e})")
check(np.allclose(V @ c, s0), "x_0 = sum c_j v_j reconstruction")
check(abs(c[0] - 1) < 1e-10, "c1 = 1 for a share vector (total population share)")
out("Why: x_k = c1 (1)^k v1 + c2 (l2)^k v2 + ...; since |l2|,|l3|,|l4| < 1 those terms -> 0, only c1 v1 = steady
state survives.")
R["eigenvalues"] = L(w, 8); R["eigenvectors_V"] = L(V, 6); R["V_inverse"] = L(Vinv, 6)
R["D"] = L(D, 8); R["reconstruction_error"] = float(rec_err); R["P10_error"] = float(err10)
R["x0_coefficients"] = L(c, 6); R["eigenvalue_powers_k30"] = [float(v ** 30) for v in w]
R["P10_direct"] = L(P10_dir, 6)
lam2 = abs(w[1]); R["lambda2_abs"] = float(lam2); R["slowest_mode_halflife_years"] = float(math.log(0.5) /
math.log(lam2))

# ------------------------------------------------------------------------------
# 5. CONVERGENCE SPEED
# ------------------------------------------------------------------------------
out("\n" + "=" * 78); out("5. CONVERGENCE SPEED"); out("=" * 78)
def dist_curve(start_share, kmax=120):
    x = np.array(start_share, float); d = []
    for k in range(kmax + 1): d.append(np.abs(x - pi).sum()); x = P @ x
    return np.array(d)                      # L1 distance (total variation x2) between shares and steady state
dist = dist_curve(s0)
first = lambda thr: int(np.argmax(dist < thr))
k1, k01 = first(0.01), first(0.001)
Cfit = dist[20] / lam2 ** 20                # asymptotic constant: d_k ~ C |lambda_2|^k
th = lambda thr: math.ceil(math.log(thr / Cfit) / math.log(lam2))
out(f"|lambda_2| = {lam2:.6f};  L1 distance d_k = sum_i |x_k,i - pi_i| (shares)")
out(f"d_0 = {dist[0]:.5f}; fitted constant C = d_20/|l2|^20 = {Cfit:.5f}")
out(f"Years until d_k < 1%   : actual {k1},  theoretical ceil(ln(0.01/C)/ln|l2|) = {th(0.01)}")
out(f"Years until d_k < 0.1% : actual {k01}, theoretical ceil(ln(0.001/C)/ln|l2|) = {th(0.001)}")
check(abs(k1 - th(0.01)) <= 2 and abs(k01 - th(0.001)) <= 2, "Actual convergence times agree with |lambda_2|^k
theory (within 2 years)")
R["convergence"] = {"d0": float(dist[0]), "C_fit": float(Cfit), "years_to_1pct_actual": k1,
"years_to_0.1pct_actual": k01,
                    "years_to_1pct_theory": th(0.01), "years_to_0.1pct_theory": th(0.001),
                    "d_at_k": {str(k): float(dist[k]) for k in [0, 1, 2, 5, 10, 20, 30]}}

# ------------------------------------------------------------------------------
# 6. MULTIPLE INITIAL DISTRIBUTIONS
# ------------------------------------------------------------------------------
starts = {"Census-based (approx.)": s0, "All in Delhi": np.array([1., 0, 0, 0]), "Uniform (25% each)":
np.ones(4) / 4,
          "All in Faridabad": np.array([0, 0, 0, 1.])}
out("\n" + "=" * 78); out("6. DIFFERENT STARTING POINTS -> SAME LIMIT"); out("=" * 78)
traj = {}; R["multi_start"] = {}
for nm, s in starts.items():
    xs = [s];
    for k in range(120): xs.append(P @ xs[-1])
    traj[nm] = np.array(xs)
    out(f"  {nm:<22} share after 120 yrs: {np.round(xs[-1], 6)}")
    check(np.abs(xs[-1] - pi).max() < 1e-5, f"Start '{nm}' converges to the steady state")
    R["multi_start"][nm] = {"start": L(s), "after_120": L(xs[-1], 8)}

# ------------------------------------------------------------------------------
# 7. SCENARIOS
# ------------------------------------------------------------------------------
out("\n" + "=" * 78); out("7. SCENARIO ANALYSIS (all SIMULATED what-if changes)"); out("=" * 78)
def set_col(M, col, new):                   # replace a column (dict region->prob) and check it sums to 1
    M = M.copy(); M[:, col] = [new[r] for r in REGIONS]; assert abs(M[:, col].sum() - 1) < 1e-12; return M
# Scenario A: Faridabad improves infrastructure -> retention 0.880 -> 0.920 and inflow rises (columns re-
balanced by hand).
PA = P.copy()
PA = set_col(PA, 0, {"Delhi": 0.945, "Gurugram": 0.020, "Noida": 0.020, "Faridabad": 0.015})   # Delhi ->
Faridabad 0.010 -> 0.015
PA = set_col(PA, 1, {"Delhi": 0.050, "Gurugram": 0.890, "Noida": 0.030, "Faridabad": 0.030})   # Gurugram ->
Faridabad 0.020 -> 0.030
PA = set_col(PA, 2, {"Delhi": 0.050, "Gurugram": 0.030, "Noida": 0.890, "Faridabad": 0.030})   # Noida ->
Faridabad 0.020 -> 0.030
PA = set_col(PA, 3, {"Delhi": 0.045, "Gurugram": 0.020, "Noida": 0.015, "Faridabad": 0.920})   # Faridabad keeps
more people
# Scenario B: job boom in Gurugram and Noida -> more movers from Delhi go there, and both retain more people.
```

```
PB = P.copy()
PB = set_col(PB, 0, {"Delhi": 0.920, "Gurugram": 0.035, "Noida": 0.035, "Faridabad": 0.010})
PB = set_col(PB, 1, {"Delhi": 0.040, "Gurugram": 0.920, "Noida": 0.025, "Faridabad": 0.015})
PB = set_col(PB, 2, {"Delhi": 0.040, "Gurugram": 0.025, "Noida": 0.920, "Faridabad": 0.015})
PB = set_col(PB, 3, {"Delhi": 0.050, "Gurugram": 0.040, "Noida": 0.030, "Faridabad": 0.880})
scen = {"Baseline": P, "A: Faridabad infrastructure": PA, "B: Gurugram/Noida job boom": PB}
R["scenarios"] = {}; rows = []
for nm, M in scen.items():
    check(np.allclose(M.sum(0), 1) and (M > 0).all() and all(0.88 <= M[i, i] <= 0.96 for i in range(n)),
f"Scenario '{nm}': valid column-stochastic, positive, retention in range")
    ev = np.linalg.eigvals(M); check(np.allclose(ev.imag, 0), f"Scenario '{nm}': eigenvalues real")
    ps = steady_eig(M); l2 = second_eig(M)
    check(ps.min() > 0.05, f"Scenario '{nm}': smallest steady share {100 * ps.min():.2f}% > 5%")
    R["scenarios"][nm] = {"P": L(M, 3), "steady_state": L(ps, 6), "lambda2_abs": float(l2),
                          "eigenvalues": L(np.sort(ev.real)[::-1], 6),
                          "change_vs_baseline_pct_points": L(100 * (ps - pi), 4)}
    rows.append([nm] + [f"{100 * v:.2f}" for v in ps] + [f"{l2:.4f}"])
    out(f"\n{nm}: P =\n{np.round(M, 3)}")
out("\n" + pd.DataFrame(rows, columns=["Scenario"] + [r + " (%)" for r in REGIONS] + ["|
lambda_2|"]).to_string(index=False))

# ------------------------------------------------------------------------------
# 8. SENSITIVITY: 1000 random perturbations (each entry multiplied by U[0.98, 1.02], then columns renormalised)
# ------------------------------------------------------------------------------
out("\n" + "=" * 78); out("8. SENSITIVITY ANALYSIS (1000 random +/-2% perturbations)"); out("=" * 78)
NSIM = 1000; sens = np.zeros((NSIM, n)); sens_l2 = np.zeros(NSIM)
for t in range(NSIM):
    Q = P * rng.uniform(0.98, 1.02, size=P.shape); Q = Q / Q.sum(axis=0, keepdims=True)
    sens[t] = steady_eig(Q); sens_l2[t] = second_eig(Q)
sm = pd.DataFrame({"Region": REGIONS, "Baseline %": 100 * pi, "Mean %": 100 * sens.mean(0), "Std %": 100 *
sens.std(0, ddof=1),
                   "Min %": 100 * sens.min(0), "Max %": 100 * sens.max(0)})
out(sm.round(3).to_string(index=False))
max_dev = float(np.max(100 * np.abs(sens - pi)))
out(f"Largest deviation of any share from baseline over all 1000 runs = {max_dev:.3f} percentage points")
out(f"|lambda_2| over runs: mean {sens_l2.mean():.4f}, min {sens_l2.min():.4f}, max {sens_l2.max():.4f}")
check(max_dev < 3.0, "Steady-state shares stay within 3 percentage points of baseline under +/-2% perturbations
(robust)")
R["sensitivity"] = {"n_runs": NSIM, "perturbation": "each entry multiplied by U[0.98,1.02], columns
renormalised",
                    "per_region": {REGIONS[i]: {"baseline_pct": float(100 * pi[i]), "mean_pct": float(100 *
sens[:, i].mean()),
                                                "std_pct": float(100 * sens[:, i].std(ddof=1)), "min_pct": float(100 *
sens[:, i].min()),
                                                "max_pct": float(100 * sens[:, i].max())} for i in range(n)},
                    "max_deviation_pct_points": max_dev,
                    "lambda2_mean": float(sens_l2.mean()), "lambda2_min": float(sens_l2.min()), "lambda2_max":
float(sens_l2.max())}

# ------------------------------------------------------------------------------
# 9b. 2x2 TOY EXAMPLE (by hand)
# ------------------------------------------------------------------------------
out("\n" + "=" * 78); out("9b. 2x2 TOY EXAMPLE  T = [[0.9, 0.2],[0.1, 0.8]]"); out("=" * 78)
T = np.array([[0.9, 0.2], [0.1, 0.8]]); tr = T.trace(); det = np.linalg.det(T)
disc = tr ** 2 - 4 * det; l_1 = (tr + math.sqrt(disc)) / 2; l_2 = (tr - math.sqrt(disc)) / 2
out(f"Characteristic equation: lambda^2 - (trace) lambda + det = lambda^2 - {tr:.1f} lambda + {det:.2f} = 0")
out(f"   => lambda = {l_1:.1f} and {l_2:.1f}")
out("lambda = 1: (T - I)v = 0 -> -0.1 a + 0.2 b = 0 -> a = 2b -> v1 = (2,1) -> normalised (2/3, 1/3)")
out("lambda = 0.7: (T - 0.7I)v = 0 -> 0.2 a + 0.2 b = 0 -> b = -a -> v2 = (1,-1)")
Tw, Tv = np.linalg.eig(T); pT = steady_eig(T)
check(np.allclose(sorted(Tw), [0.7, 1.0]), "Toy 2x2: eigenvalues are 1 and 0.7")
check(np.allclose(pT, [2 / 3, 1 / 3]), "Toy 2x2: steady state is (2/3, 1/3)")
xt = np.array([0.5, 0.5]); toy_seq = [xt.tolist()]
for k in range(3): xt = T @ xt; toy_seq.append(xt.tolist())
out("Start (0.5,0.5): " + " -> ".join(str(np.round(v, 4)) for v in toy_seq))
R["toy_2x2"] = {"T": T.tolist(), "eigenvalues": [l_1, l_2], "steady_state": pT.tolist(),
"sequence_from_(0.5,0.5)": toy_seq,
                "v1": [2, 1], "v2": [1, -1]}

# ------------------------------------------------------------------------------
# 10. LIMITATIONS: NON-REGULAR / ABSORBING / PERIODIC
```

```
# ------------------------------------------------------------------------------
out("\n" + "=" * 78); out("10. WHAT IF P IS NOT REGULAR?"); out("=" * 78)
Pabs = np.array([[0.8, 0.1, 0.0], [0.2, 0.8, 0.0], [0.0, 0.1, 1.0]])        # state 3 has retention 1
(absorbing)
check(np.allclose(Pabs.sum(0), 1), "Absorbing example is column-stochastic")
reg_abs = any((np.linalg.matrix_power(Pabs, k) > 0).all() for k in range(1, 200))
check(not reg_abs, "Absorbing example is NOT regular (no power of it is strictly positive)")
xa = np.array([1., 0, 0]); xa = np.linalg.matrix_power(Pabs, 200) @ xa
out(f"Absorbing chain, start in state 1, after 200 steps: {np.round(xa, 6)}  -> everybody ends in the absorbing
state 3")
Pper = np.array([[0., 1.], [1., 0.]])
out("Periodic chain [[0,1],[1,0]]: eigenvalues " + str(np.linalg.eigvals(Pper)) + "; start (1,0) -> " +
    " , ".join(str((np.linalg.matrix_power(Pper, k) @ np.array([1., 0])).tolist()) for k in range(1, 5)) + "
(flips forever, no limit)")
R["non_regular"] = {"absorbing_P": Pabs.tolist(), "absorbing_limit_from_state1": L(xa, 6),
                    "absorbing_eigenvalues": L(np.sort(np.linalg.eigvals(Pabs).real)[::-1], 6),
                    "periodic_P": Pper.tolist(), "periodic_eigenvalues": [1.0, -1.0]}

# ------------------------------------------------------------------------------
# FIGURES
# ------------------------------------------------------------------------------
# fig1 heatmap
fig, ax = plt.subplots(figsize=(7, 5.8))
lab_f = ["from " + r for r in REGIONS]; lab_t = ["to " + r for r in REGIONS]
if sns is not None:
    sns.heatmap(P, annot=True, fmt=".3f", cmap="viridis", xticklabels=lab_f, yticklabels=lab_t, ax=ax,
cbar_kws={"label": "Probability"}, annot_kws={"size": 12})
else:
    im = ax.imshow(P, cmap="viridis"); plt.colorbar(im, label="Probability"); ax.set_xticks(range(n));
ax.set_yticks(range(n))
    ax.set_xticklabels(lab_f); ax.set_yticklabels(lab_t)
    for i in range(n):
        for j in range(n): ax.text(j, i, f"{P[i, j]:.3f}", ha="center", va="center", color="w")
ax.set_title("Fig 1: Transition matrix P (columns sum to 1)"); ax.set_xlabel("Region this year (column j)");
ax.set_ylabel("Region next year (row i)")
ax.grid(False); plt.xticks(rotation=30); savefig("fig1_transition_heatmap.png")

# fig2 flow diagram (matplotlib only)
pos = {"Delhi": (0, 1), "Gurugram": (-1.4, -0.6), "Noida": (1.4, -0.6), "Faridabad": (0, -1.8)}
fig, ax = plt.subplots(figsize=(8.5, 7.5)); ax.set_aspect("equal"); ax.axis("off"); ax.grid(False)
for i, ri in enumerate(REGIONS):                    # arrow from j (source) to i (target), width ~ P[i,j]
    for j, rj in enumerate(REGIONS):
        if i == j: continue
        (xa_, ya_), (xb_, yb_) = pos[rj], pos[ri]
        ar = FancyArrowPatch((xa_, ya_), (xb_, yb_), connectionstyle="arc3,rad=0.18", arrowstyle="-|>",
mutation_scale=14 + 220 * P[i, j],
                             lw=0.6 + 90 * P[i, j], color=COLORS[rj], alpha=0.75, shrinkA=34, shrinkB=34)
        ax.add_patch(ar)
        dx, dy = xb_ - xa_, yb_ - ya_; mx, my = (xa_ + xb_) / 2 + 0.5 * 0.18 * dy, (ya_ + yb_) / 2 - 0.5 * 0.18
* dx   # midpoint of the curved arrow
        ax.text(mx, my, f"{P[i, j]:.3f}", fontsize=9, color="k", ha="center", va="center", zorder=7,
                bbox=dict(boxstyle="round,pad=0.15", fc="white", ec="none", alpha=0.9))
for r, (px, py) in pos.items():
    ax.scatter([px], [py], s=5200, color=COLORS[r], edgecolor="k", zorder=5)
    ax.text(px, py + 0.07, r, ha="center", va="center", fontsize=11, fontweight="bold", color="white" if r in
("Delhi", "Noida") else "black", zorder=6)
    ax.text(px, py - 0.14, f"stay {P[REGIONS.index(r), REGIONS.index(r)]:.3f}", ha="center", va="center",
fontsize=9, color="white" if r in ("Delhi", "Noida") else "black", zorder=6)
ax.set_xlim(-2.4, 2.4); ax.set_ylim(-2.6, 1.8)
ax.set_title("Fig 2: Yearly movement between regions (arrow thickness ~ probability; colour = origin)");
savefig("fig2_flow_diagram.png")

# fig3 shares over time
fig, axs = plt.subplots(1, 2, figsize=(13, 5)); yrs = np.arange(K + 1)
for i, r in enumerate(REGIONS):
    axs[0].plot(yrs, 100 * S[:, i], color=COLORS[r], lw=2.2, label=r); axs[0].axhline(100 * pi[i],
color=COLORS[r], ls=":", lw=1)
axs[0].set_title("Fig 3a: Population share over time (dotted = steady state)"); axs[0].set_xlabel("Years k");
axs[0].set_ylabel("Share of total population (%)"); axs[0].legend()
axs[1].stackplot(yrs, 100 * S.T, labels=REGIONS, colors=REG_COL, alpha=0.9)
```

```
axs[1].set_title("Fig 3b: Stacked area of shares"); axs[1].set_xlabel("Years k"); axs[1].set_ylabel("Share
(%)"); axs[1].set_xlim(0, K); axs[1].set_ylim(0, 100); axs[1].legend(loc="center right")
savefig("fig3_population_over_time.png")

# fig4 initial vs steady
fig, ax = plt.subplots(figsize=(8.5, 5)); xx = np.arange(n); wd = 0.38
b1 = ax.bar(xx - wd / 2, 100 * s0, wd, color=REG_COL, edgecolor="k", label="Initial (Census-based)")
b2 = ax.bar(xx + wd / 2, 100 * pi, wd, color=REG_COL, edgecolor="k", hatch="//", label="Steady state")
for bars in (b1, b2):
    for b in bars: ax.text(b.get_x() + b.get_width() / 2, b.get_height() + 0.8, f"{b.get_height():.1f}%",
ha="center", fontsize=10)
ax.set_xticks(xx); ax.set_xticklabels(REGIONS); ax.set_ylabel("Share of total population (%)");
ax.set_title("Fig 4: Initial vs steady-state distribution")
ax.legend(); ax.set_ylim(0, 85); savefig("fig4_initial_vs_steady.png")

# fig5 convergence
fig, ax = plt.subplots(figsize=(8.5, 5)); kk = np.arange(len(dist))
ax.semilogy(kk, dist, "o-", ms=3, color="#0072B2", label="Actual distance $d_k$ (L1)")
ax.semilogy(kk, Cfit * lam2 ** kk, "--", color="#D55E00", lw=2, label=f"Theory $C|\\lambda_2|^k$, $|\\lambda_2|
$={lam2:.4f}")
ax.axhline(0.01, color="gray", ls=":", label="1% threshold"); ax.axhline(0.001, color="k", ls=":", label="0.1%
threshold")
ax.set_ylim(1e-8, 2); ax.set_xlim(0, 120); ax.set_xlabel("Years k"); ax.set_ylabel("Distance to steady state
(log scale)")
ax.set_title("Fig 5: Convergence to steady state (log scale)"); ax.legend(); savefig("fig5_convergence.png")

# fig6 several starts
fig, axs = plt.subplots(2, 2, figsize=(12, 8)); mk = ["-", "--", "-.", ":"]; scol = ["#000000", "#D55E00",
"#56B4E9", "#666666"]
for i, r in enumerate(REGIONS):
    ax = axs.flat[i]
    for (nm, tr_), ls, cc in zip(traj.items(), mk, scol): ax.plot(100 * tr_[:81, i], ls=ls, color=cc, lw=2,
label=nm)
    ax.axhline(100 * pi[i], color=COLORS[r], lw=3, alpha=0.5, label="Steady state")
    ax.set_title(f"{r}"); ax.set_xlabel("Years k"); ax.set_ylabel("Share (%)")
axs.flat[0].legend(fontsize=8); fig.suptitle("Fig 6: Different starting points converge to the same steady
state", fontsize=14); savefig("fig6_multiple_starts.png")

# fig7 scenarios
fig, ax = plt.subplots(figsize=(10, 5.5)); wd = 0.26; hat = ["", "//", ".."]
for s_i, (nm, M) in enumerate(scen.items()):
    ps = steady_eig(M)
    bars = ax.bar(xx + (s_i - 1) * wd, 100 * ps, wd, color=REG_COL, edgecolor="k", hatch=hat[s_i], label=nm)
    for b in bars: ax.text(b.get_x() + b.get_width() / 2, b.get_height() + 0.6, f"{b.get_height():.1f}",
ha="center", fontsize=8)
ax.set_xticks(xx); ax.set_xticklabels(REGIONS); ax.set_ylabel("Steady-state share (%)"); ax.set_title("Fig 7:
Steady state under scenarios (hatch pattern = scenario)")
from matplotlib.patches import Patch
ax.legend(handles=[Patch(fc="white", ec="k", hatch=hat[i], label=nm) for i, nm in enumerate(scen)]);
savefig("fig7_scenarios.png")

# fig8 sensitivity box plot (one panel per region so each has its own readable scale)
fig, axs = plt.subplots(1, n, figsize=(13, 5))
for i, r in enumerate(REGIONS):
    ax = axs[i]; bp = ax.boxplot([100 * sens[:, i]], patch_artist=True, widths=0.5, medianprops=dict(color="k",
lw=2))
    bp["boxes"][0].set_facecolor(COLORS[r]); bp["boxes"][0].set_alpha(0.9)
    ax.scatter([1], [100 * pi[i]], marker="D", color="k", zorder=5, label="Baseline")
    ax.set_xticks([1]); ax.set_xticklabels([r]); ax.set_ylabel("Steady-state share (%)" if i == 0 else "")
    if i == 0: ax.legend()
fig.suptitle("Fig 8: Sensitivity of steady-state shares to +/-2% perturbations (1000 runs)", fontsize=13);
savefig("fig8_sensitivity_boxplot.png")

# fig9 eigenvalues on real line
fig, ax = plt.subplots(figsize=(9, 3.2)); ax.axhline(0, color="k", lw=1); ax.axvspan(-1, 1, color="#56B4E9",
alpha=0.15, label="Unit interval (-1, 1)")
for j in range(n):
    ax.scatter(w[j], 0, s=140, color="#D55E00" if j == 0 else "#0072B2", zorder=5, edgecolor="k")
    ax.annotate(f"$\\lambda_{j + 1}$={w[j]:.4f}", (w[j], 0), xytext=(0, 18 + 16 * (j % 2)), textcoords="offset
points", ha="center", fontsize=10)
```

```
ax.set_xlim(0.84, 1.03); ax.set_ylim(-0.5, 0.8); ax.set_yticks([]); ax.set_xlabel("Real axis");
ax.set_title("Fig 9: Eigenvalues of P on the real line (only $\\lambda_1$ = 1 touches the boundary)");
ax.legend(loc="lower left")
savefig("fig9_eigenvalues.png")

# ------------------------------------------------------------------------------
# WRITE OUTPUTS
# ------------------------------------------------------------------------------
R["assertions_passed"] = PASSED; R["note"] = "Transition probabilities are SIMULATED; populations are
APPROXIMATE (Census 2011, to be verified)."
R["scenario_table_rows"] = rows
(OUT / "results.json").write_text(json.dumps(R, indent=2))
out("\n" + "=" * 78); out(f"ASSERTIONS PASSED: {len(PASSED)}"); out("Figures saved in: " + str(FIG))
(OUT / "results.txt").write_text("\n".join(LOG))
```

Appendix B: Hand-Calculation Sheet

B.1 Population of Gurugram after one year (use a calculator)

x_1[Gurugram] = Σ_j P[Gurugram][j] · x_0[j] (row 2 of P times the initial vector).

From region j	P[Gurugram][j]	x_0[j]	Product
Delhi	0.020	16,787,941	335,758.8
Gurugram	0.890	1,514,085	1,347,535.6
Noida	0.030	1,674,714	50,241.4
Faridabad	0.030	1,809,733	54,292.0
Total			**1,787,827.9**

Check: the same row sum is the second entry of P x_0 in Python.

B.2 Eigenvalues of P using trace and structure

The trace of P is 3.610, and the eigenvalues add up to the trace: 1.0000 + 0.8862 + 0.8638 + 0.8600 = 3.610. This is a quick check that the printed eigenvalues are right.

B.3 The 2 × 2 example in full

1. T = [[0.9, 0.2],[0.1, 0.8]]. Trace = 1.7 and determinant = 0.9×0.8 − 0.2×0.1 = 0.70.
2. Characteristic equation: $\lambda^2 - 1.7\lambda + 0.70 = 0$, i.e. $(\lambda - 1)(\lambda - 0.7) = 0$.
3. $\lambda = 1$: solve $(T - I)v = 0$. Row 1: $-0.1a + 0.2b = 0 \Rightarrow a = 2b$. Take $v_1 = (2, 1)$; divide by the sum 3 to get $\pi = (2/3, 1/3)$.
4. $\lambda = 0.7$: $(T - 0.7I)v = 0$. Row 1: $0.2a + 0.2b = 0 \Rightarrow b = -a$. Take $v_2 = (1, -1)$.
5. V = [[2, 1],[1, −1]], det V = −3, V^{-1} = [[1/3, 1/3],[1/3, −2/3]], D = diag(1, 0.7).
6. $T^n = V D^n V^{-1}$. As $n \to \infty$, $D^n \to$ diag(1, 0), so $T^n \to$ [[2/3, 2/3],[1/3, 1/3]]: every column is π.
7. Start $x_0 = (0.5, 0.5)$: $x_1 = (0.9\times0.5 + 0.2\times0.5, 0.1\times0.5 + 0.8\times0.5) = (0.55, 0.45)$. Then x_2 = (0.585, 0.415) and x_3 = (0.6095, 0.3905), approaching (0.6667, 0.3333).

B.4 Steady state of P by hand (outline)

Solve $(P - I)\pi = 0$ with $\pi_1 + \pi_2 + \pi_3 + \pi_4 = 1$: write the augmented matrix, replace the last equation by the row of ones, and apply row operations (forward elimination, then back-substitution). The program prints the row-echelon form it obtains in results.txt (Section 3), so each row operation can be checked against it.